

Resolviendo VRPB de forma Greedy + HC-BI

Estado de avance del código y explicación de la implementación actual.

Alvaro Opazo
Rol: 201973550-0
Ayudante: Nangel Coello
Junio de 2025.



Contexto del problema

Una breve explicación del problema a abordar antes de la explicación

Origen del problema

Goetschalckx & Jacobs-Blecha (1989), Presentan formalmente el VRPB, donde primero se entregan productos (linehaul) y luego se recogen (backhaul). Destacan su dificultad computacional y aplicaciones reales.



Complejidad y Relevancia

Koç & Laporte (2018), Revisión clave que muestra cuán amplio y desafiante es el VRPB. Propone usar heurísticas/metaheurísticas por el enorme espacio de búsqueda.



Apoyo a la Técnica

Osman & Kelly (1996), Justifican el uso de Hill Climbing como mejora local. Refuerzan que combinarlo con Greedy es útil en problemas complejos.



Explicación general del código:

Parser(leerInstancia)

Lee la instancia desde un archivo .txt y construye objetos Nodo

Evaluación(evaluarSolucion, evaluarRuta)

Calcula la distancia total recorrida (Función Objetivo)

Restricciones(esSolucion Factible)

Verifica que se cumpla capacidad y orden linehaul-backhaul

Greedy(generarSolucion Greedy)

Construye rutas eligiendo el cliente más cercano factible (Heurística del más cercano)

HC-BI(hillClimbingBI)

Busca el mejor movimiento entre todas las rutas que reduzca el costo total

Salida(escribirSalida)

Formato de archivo con rutas, distancia total y métricas clave

Función de evaluación

Qué hace?

Calcula la distancia total recorrida por todos los vehículos en la solución actual

Cómo lo hace?

1. Para cada ruta, suma las distancias euclidianas entre cada par consecutivo de nodos
2. Calcula la demanda total atendida por cada vehículo
3. El resultado final es la suma de todas las distancias individuales de las rutas: $\text{valorFuncion} = \sum \text{distancia}(\text{ruta}_i)$

```
SolucionVRPB evaluarSolucion(vector<Ruta>&
rutas, const vector<Nodo>& nodos)
{
    SolucionVRPB solucion;
    solucion.rutas = rutas;
    solucion.valorFuncion = 0.0;

    for (Ruta& ruta : solucion.rutas) {
        ruta.distancia = evaluarRuta(ruta,nodos);
        ruta.demandaTotal = 0.0;
        for (int id : ruta.nodos)
            ruta.demandaTotal +=
                nodos[id - 1].demanda;
        solucion.valorFuncion += ruta.distancia;
    }

    return solucion;
}
```

```
vector<Ruta> generarSolucionesGreedy(const  
instanciaVRPB& instancia)
```

```
{
```

```
    Para cada vehículo:
```

```
        Inicializar ruta desde el depósito
```

```
        Inicializar carga = 0
```

```
    Mientras queden clientes linehaul posibles:
```

```
        Elegir el más cercano y factible
```

```
        Agregar a la ruta y actualizar carga
```

```
    Mientras queden clientes backhaul posibles:
```

```
        Elegir el más cercano y factible
```

```
        Agregar a la ruta y actualizar carga
```

```
    Cerrar la ruta volviendo al depósito
```

```
    Guardar ruta
```

```
}
```

Heurística Greedy Inicial

Cómo funciona?

1. Se parte del depósito (nodo 1)
2. Se selecciona el **cliente linehaul más cercano** que pueda ser atendido (capacidad no superada)
3. Una vez que no queden linehails viables, se procede a atender **clientes backhails**, también por cercanía y factibilidad
4. La ruta se cierra volviendo al depósito
5. Se repite el proceso con nuevos vehículos hasta cubrir todos los clientes posibles

Limitación

No garantiza la mejor cobertura ni solución óptima

Hill Climbing - Best Improvement

Propósito

Mejorar la solución inicial buscando movimientos entre rutas que reduzcan el costo total

Cómo se aplica?

1. Se exploran **todos los movimientos posibles** de un cliente desde su ruta actual hacia otra
2. Se calcula el costo de la solución resultante
3. Si se encuentra una solución **mejor y factible**, se acepta
4. El proceso se repite hasta que no hay más mejoras

```
SolucionVRPB hillClimbingBI(const solucionVRPB&
solucionInicial, const InstanciaVRPB& instancia)
```

```
{
    Inicializar solución actual = solución inicial
    Repetir:

    Buscar el mejor vecino entre todas las
    soluciones con un solo movimiento

    Si mejora la función de evaluación:
        Actualizar la solución actual
    Si no hay mejora:
        Terminar
    Hasta que no haya mejoras
    Retornar la mejor solución encontrada
}
```

```
SolucionVRPB hillClimbingBI(const solucionVRPB&  
solucionInicial, const InstanciaVRPB& instancia)
```

```
{  
...
```

```
Para cada ruta i:
```

```
    Para cada cliente j en la ruta i:
```

```
        Para cada ruta k distinta de i:
```

```
            Para cada posición válida en ruta k:
```

```
                Intentar mover cliente j desde  
                ruta i a ruta k
```

```
                Si la solución resultante es  
                factible:
```

```
                    Evaluar solución y guardar si  
                    es mejor
```

```
}
```

Movimiento

Qué hace?

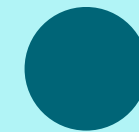
1. Toma **un cliente** desde una posición dentro de una ruta *i*
2. Lo **inserta en otra ruta *k*** (no igual a *i*) en todas las **posiciones posibles**
3. Evalúa si el cambio:
 - Genera una solución **factible** (capacidad y orden linehaul-backhaul)
 - **Mejora el valor de la función objetivo** (menor distancia total)
4. Si se cumple todo lo anterior, se actualiza la solución

Tipo de movimiento

Se llama reubicación (relocate), es clásico en **VRP** y **VRPB**

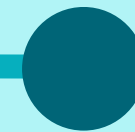
Funcionamiento general del código

PASO 1



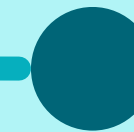
Se lee una instancia desde archivo

PASO 2



Se genera una solución inicial greedy

PASO 3



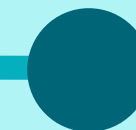
Se evalúa con la función objetivo (distancia total)

PASO 4



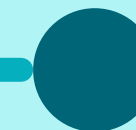
Se aplica Hill Climbing para mejorarla

PASO 5



Se verifica la factibilidad final

PASO 6



Se escribe la solución en un archivo .out con todos los datos

Conclusiones

Implementación funcional

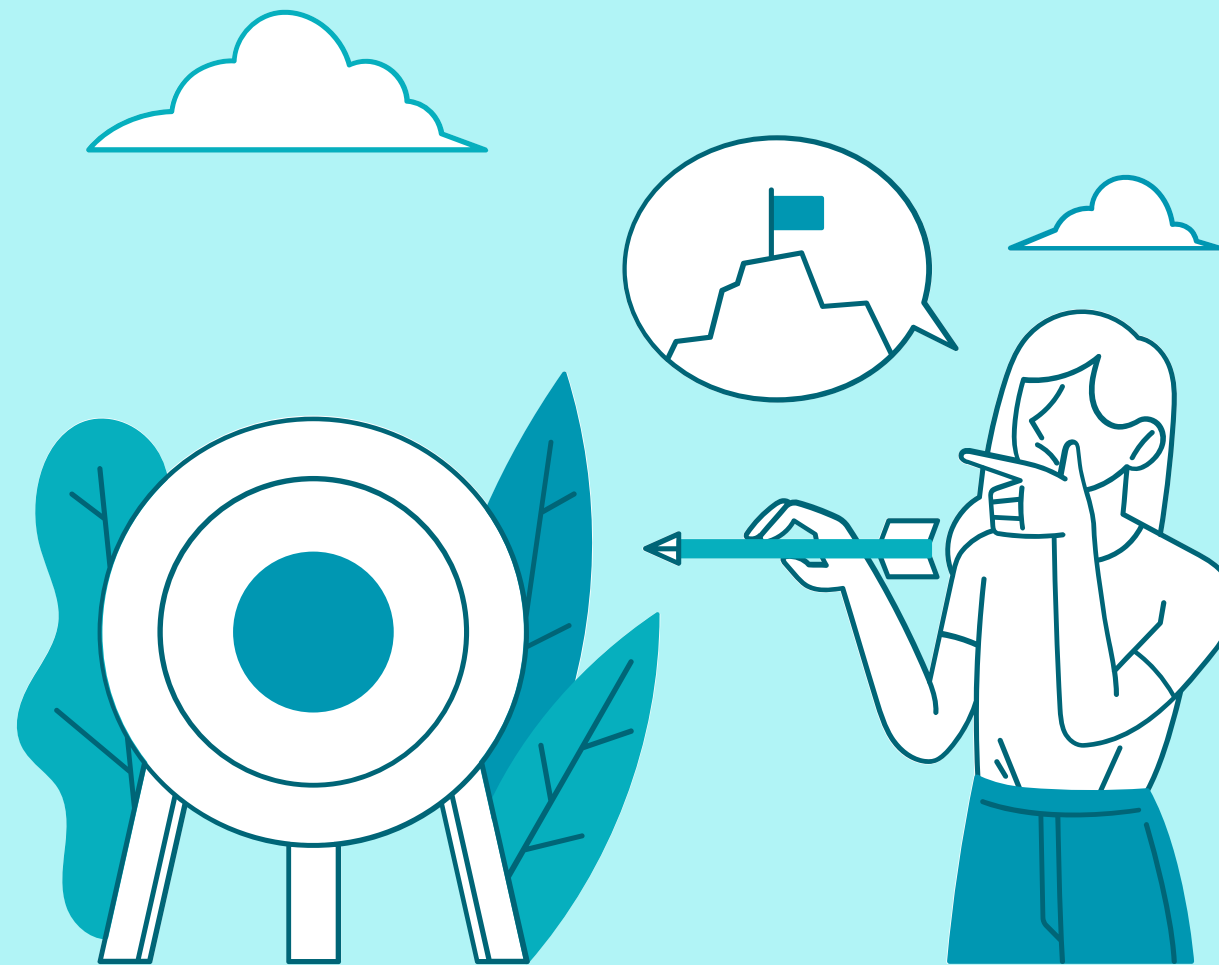
Actualmente se cuenta con una implementación basada en la combinación de una heurística Greedy + HB-BI

Cobertura de requerimientos mínimos

El código cumple con los aspectos solicitados, todos correctamente integrados

Estructura clara y modular

Se diseñaron estructuras específicas para representar nodos, rutas y soluciones, facilitando la comprensión y modificación



Generación incompleta de soluciones

La solución greedy no garantiza cobertura total de clientes, lo que provoca soluciones no factibles

Complejidad en validación y movimientos

Durante la validación se identificaron errores sutiles relacionados con el orden linehaul/backhaul y la gestión de capacidad.

Dilemas de diseño y estrategia de mejora

Se enfrentaron decisiones abiertas como: qué tipo de movimiento usar, cómo definir el vecindario en HC-BI, y si se debía aceptar soluciones infactibles como punto de partida o descartarlas

**¡GRACIAS POR
ESCUCHAR!**

Alguna duda?

